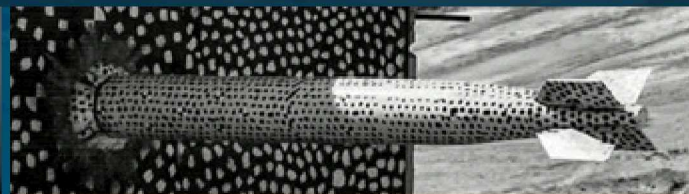


SAND2019-10616C

The GR PDU Utilities



PRESENTED BY

Jacob A Gilbert – at the 2019 GNU Radio Conference

jacob.gilbert@sandia.gov

SAND2019-XXXX

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

2 Presentation Summary

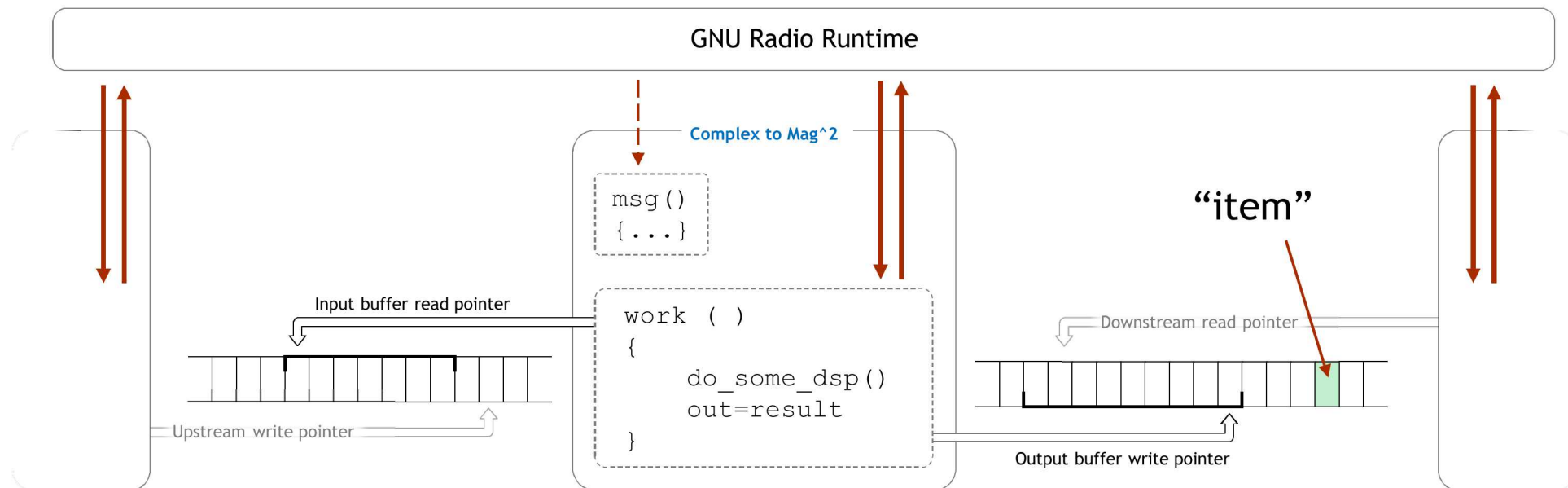
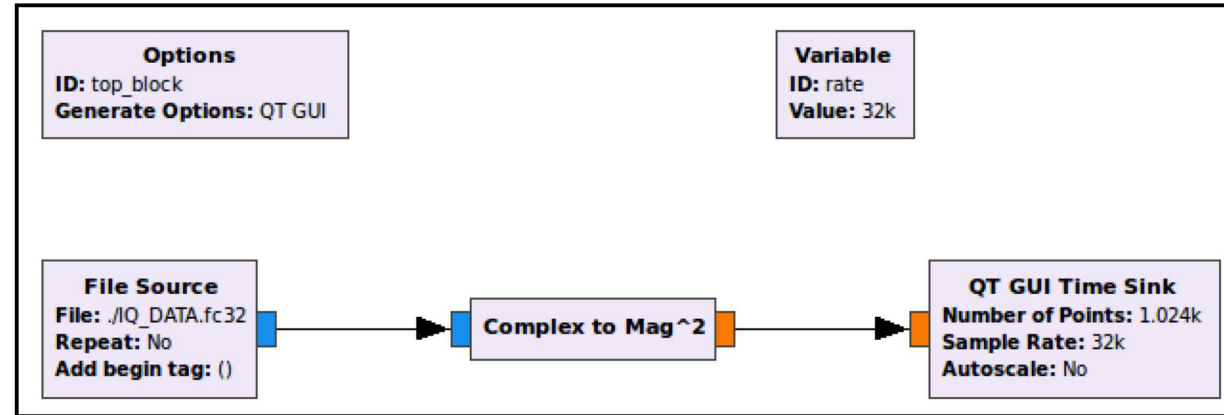
Objective:

Provide an overview the gr-pdu_utils module and associated Sandia GR software.

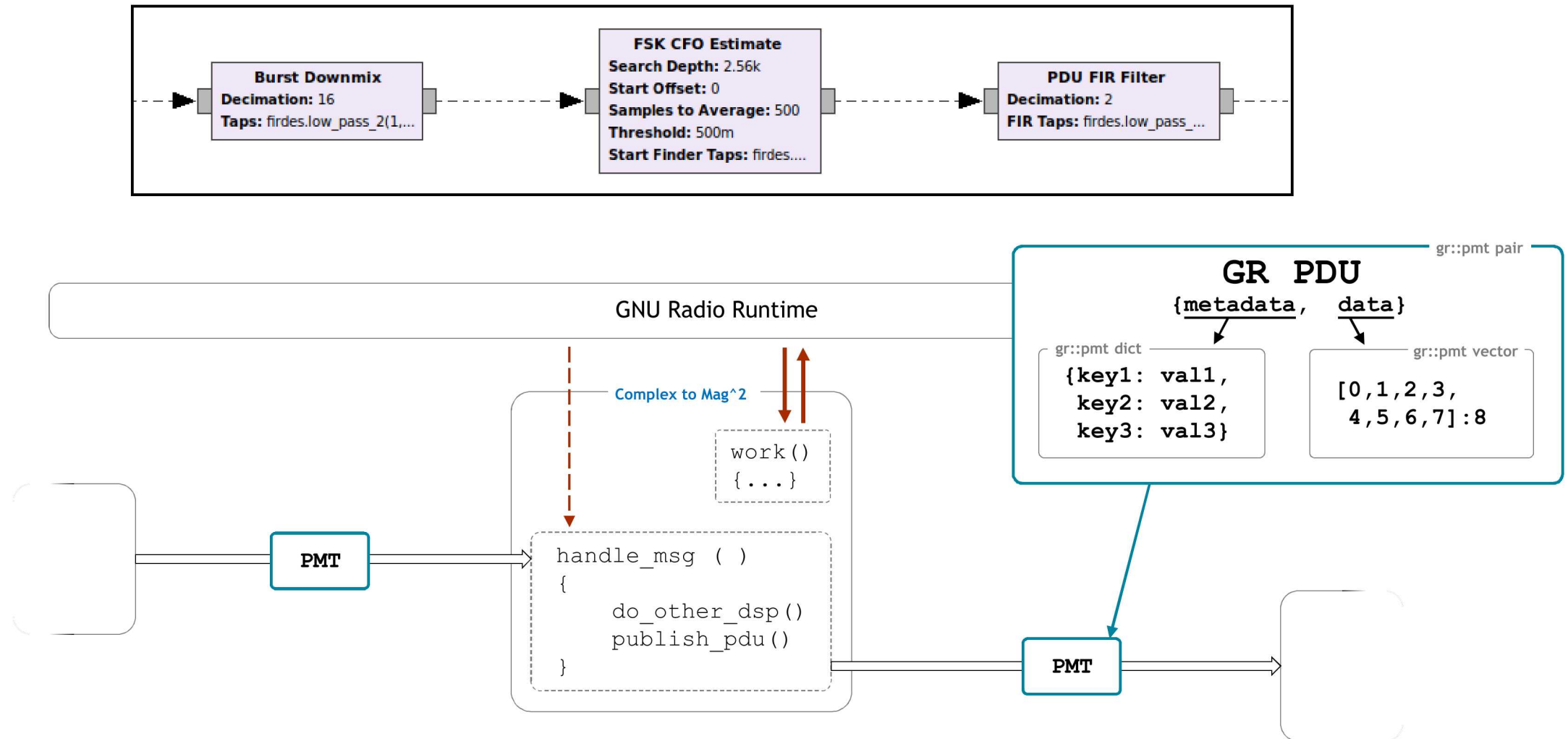
Content:

- Motivation for the PDU Utilities
- Overview of Existing Tools
- Theory of Operation
- Other Sandia Modules
- Concluding Remarks

3 GR Operation: Streaming

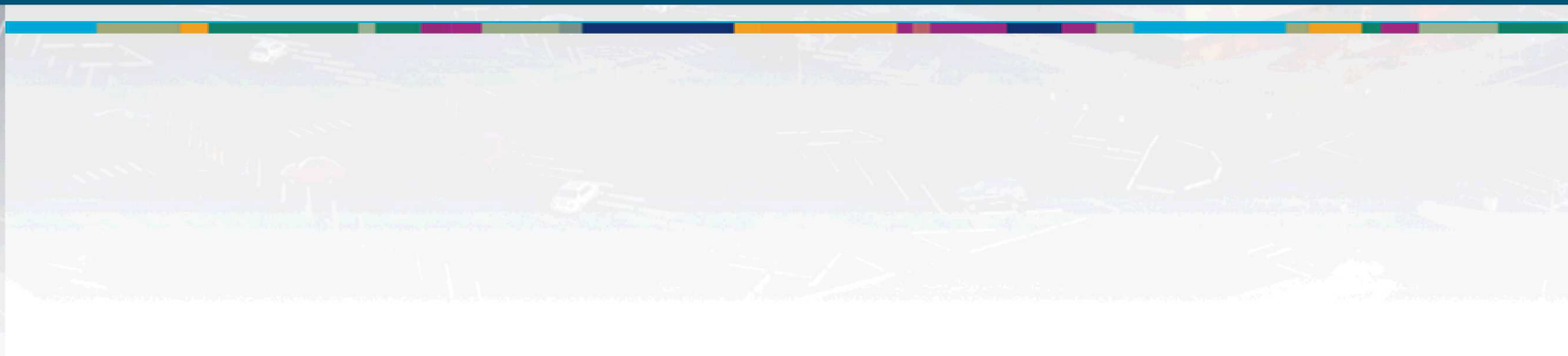
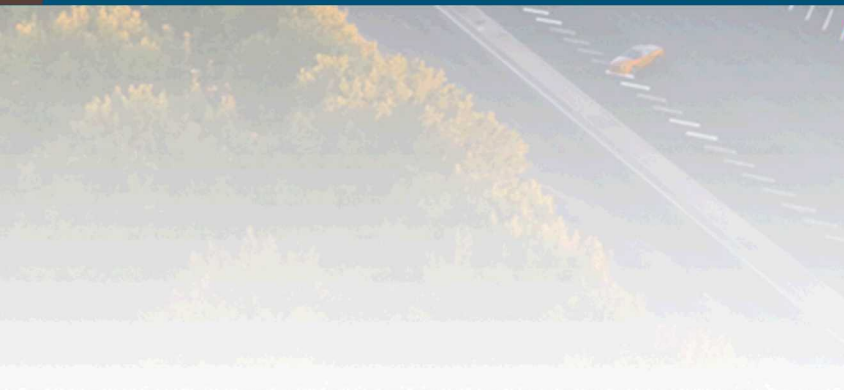


GR Asynchronous Message Passing / PDUs





Motivation



Motivation for the PDU Utilities

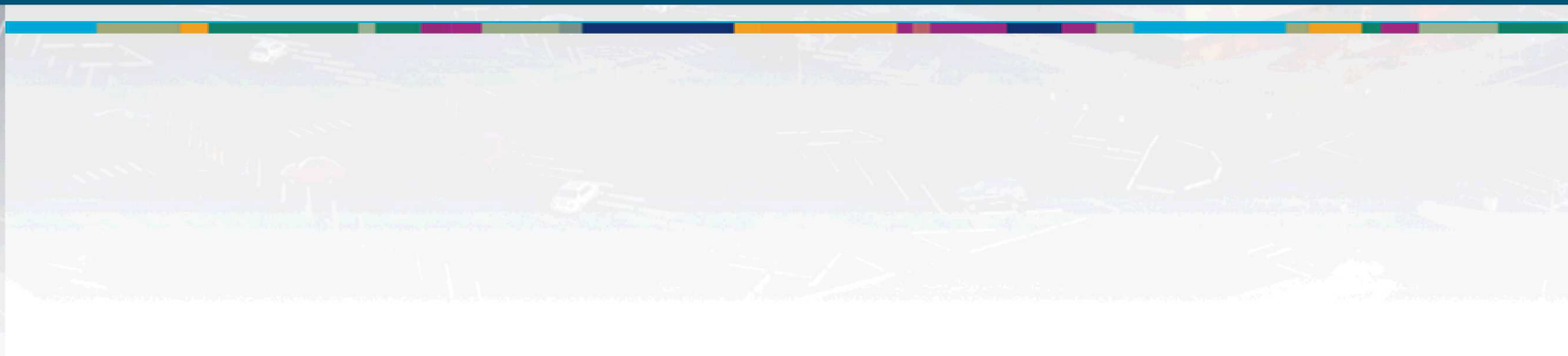
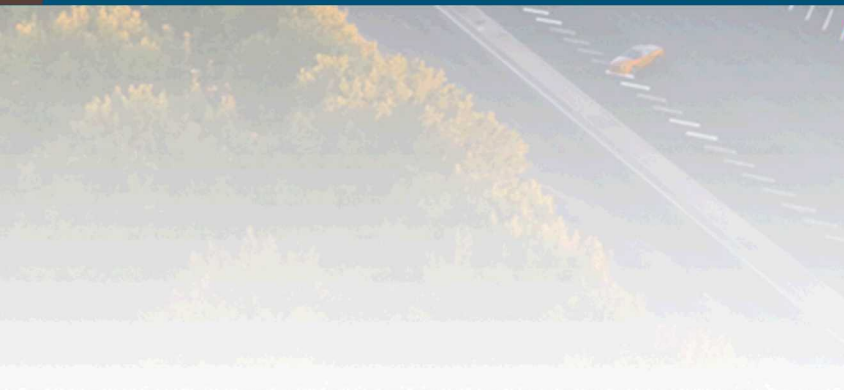
- Receive a burst of data and respond with new burst based on the received data
- Current tools exist but have limitations:
 - gr-uhd Stream Tags
 - Tagged Stream Blocks
 - GR Packet Communications API
 - gr-eventstream
- Complicated by real-world radio protocol constraints:
 - Minimum RF turnaround time (latency)
 - Other TDMA system constraints such as relative burst timing
 - Frequency agility, RF tuning and tracking
- ‘We tried GNU Radio but there was too much latency so we used $\left[\begin{array}{c} \text{libuhd} \\ \text{C++} \\ \text{RFNoC} \\ \text{X-MIDAS} \end{array} \right]$ instead’

7 Required Capabilities

- RX: Convert received bursts of modulated data into PDUs for further processing
 - ~Sample accurate time estimate of bursts
 - Fixed, multi, arbitrary frequency operation
- TX: Convert PDUs into OTA bursts of modulated data for HW interaction
 - ~Sample-accurate transmission time
 - Fixed, multi, arbitrary frequency operation
 - Minimize round-trip latency through SDR system
- Portable: Minimize specialized hardware dependencies
- Robustness: Quickly recover from overflows and dropped data
- Efficient: Design for lower power embedded hardware



Existing Toolset

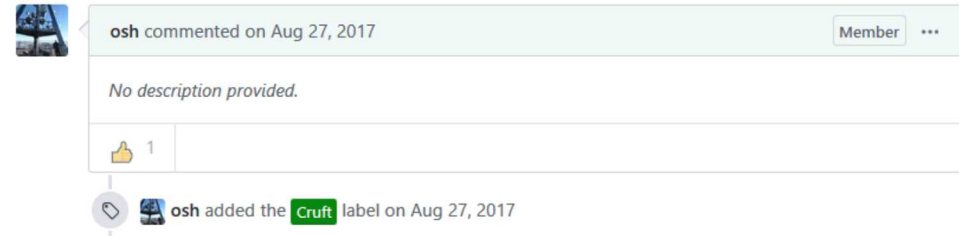


9 Existing In Tree Tools

- GR Tagged Stream API
 - Support for burst processing within the streaming architecture of GR
 - Going to be deprecated?

remove tagged stream blocks #1446

 Open osh opened this issue on Aug 27, 2017 · 4 comments

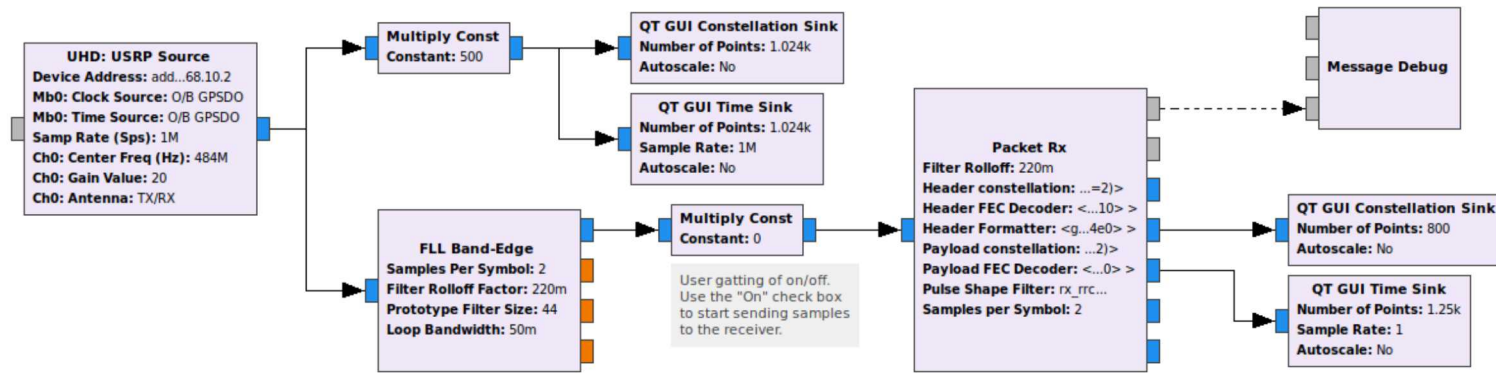


“there's a lot of bugs and inefficiencies, and quite some grief¹ and undocumented contracts connected to TSBs” - M. Müller, 2017

- gr-uhd Stream Tags
 - ‘rx_time’ provides for RX time tracking, ‘tx_time’ allows synchronized, timed transmissions
 - ‘tx_sob’ and ‘tx_eob’ tags allow for single port half-duplex transceiver
 - Not a complete solution, still streaming based
 - Since this relies on hardware, simulation can be difficult (DIY emulators)
 - Extremely useful though – leveraged by gr-pdu_utils and extended to Sidekiq hardware

Existing In Tree Tools: Packet Communications API

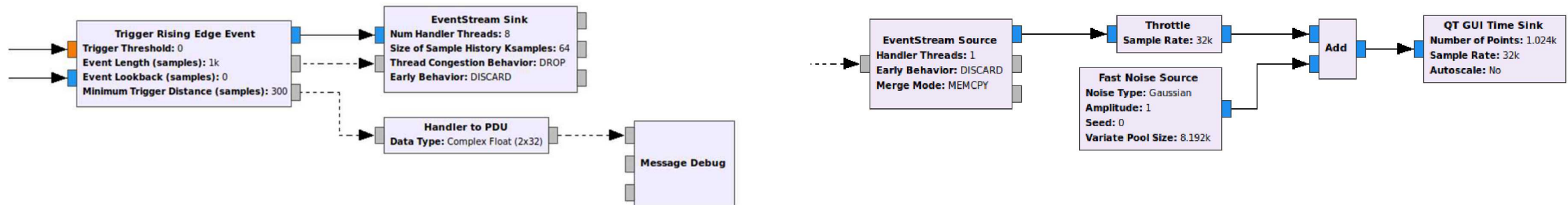
- Powerful packet processing API, somewhat complicated (makes use of TSBs...)



- Appears to use open loop timing, lots of rework to meet our requirements
- Doxygen Documentation: https://www.gnuradio.org/doc/doxygen/page_packet_comms.html
- Interesting Module Example: <https://www.gnuradio.org/deptofdefense/gr-uaslink>
- Reasonable option for custom protocols and more complicated modulations
- We found the PDU Utilities to be easier for existing, simple protocols

OOT Modules: gr-eventstream

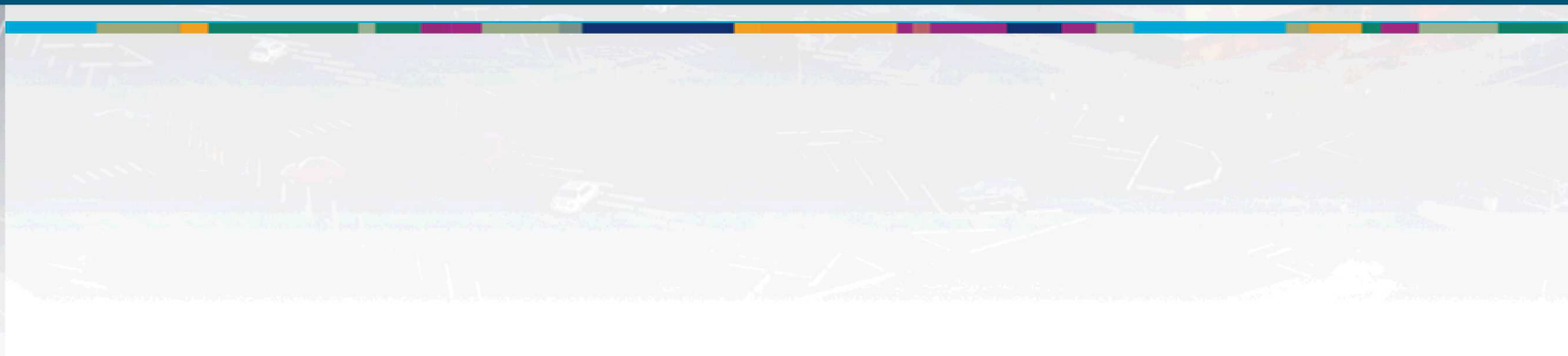
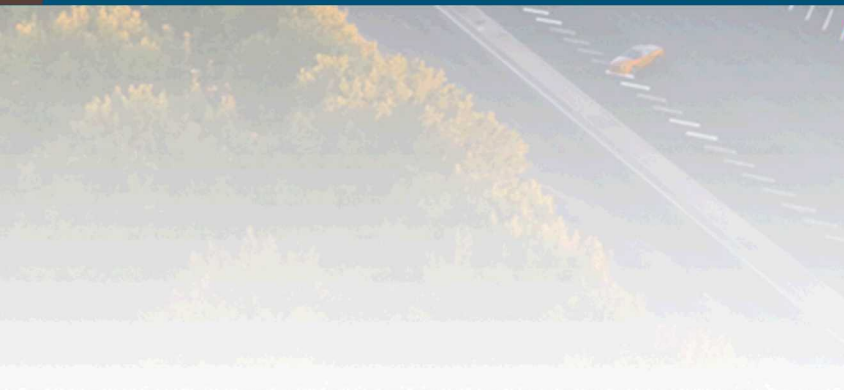
- Allows burst processing with closed loop timing by interleaving bursts into TX stream
- Relies on GR backpressure which means all buffers will impact latency
 - Minimize number of streaming blocks between ES source and HW Sink
- Good description of functionality: <https://oshearesearch.com/index.php/tag/eventstream/>



- Limitations for our use: T/R latency, fairly different from our current approach
 - Possible that we will integrate ES concepts in future

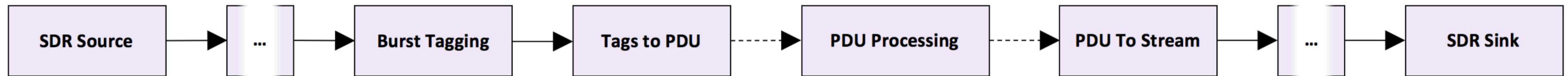


Theory of Operation



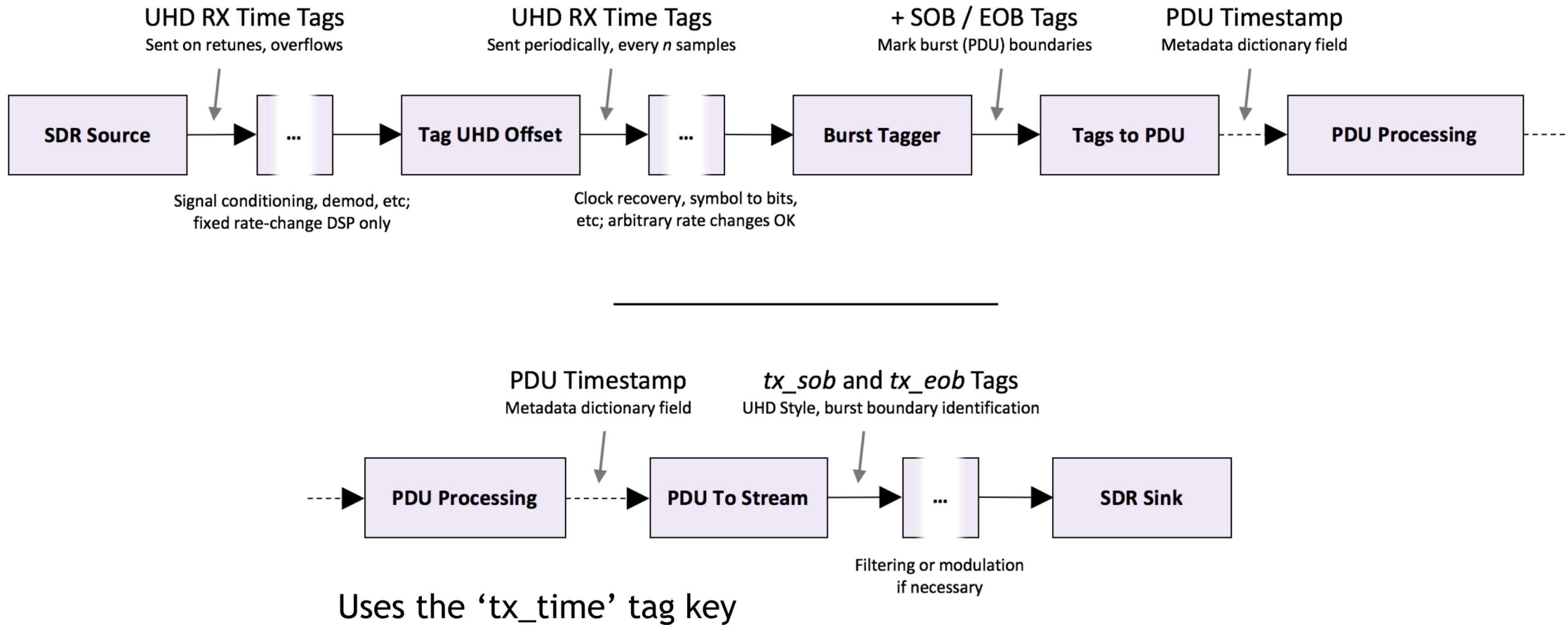
13 Usage of the PDU Utilities

- Robust set of blocks for converting between streaming and async message (PDU)



- Receive energy, basic preprocessing
- Identify start/stop of burst
- Convert required data elements to PDU
- Keep track of time
- Do higher layer processing
- Convert PDU to stream if required
- Emit RF energy at specified time

PDU Utilities Time Management

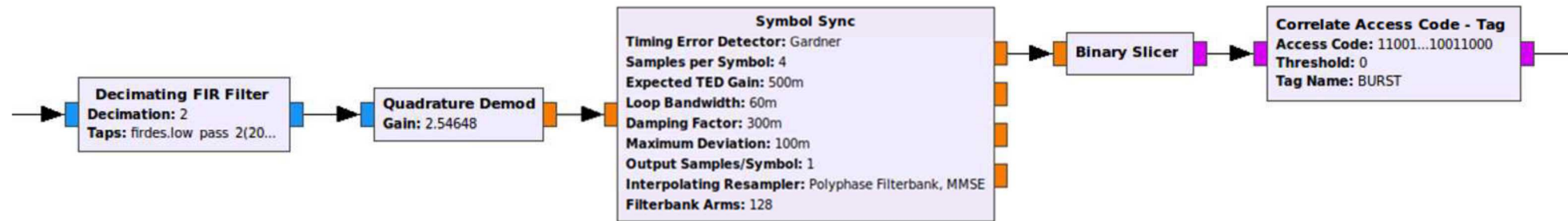


The Timing Utilities Module

- Separated from gr-pdu_utils to contain various time tracking functions
 - Some complexity is now OBE (thank you awalls)
- Lots of time tuple related code
 - Tracking UHD Time Tuples
 - Adding periodic Time Tuples
- Does this need to be a separate module?
 - No, BUT...

UHD Time Tuple:	(rx_time {1432 0.912341})
	[Key] [Integer Secs] [Fractional Secs] [Sample Offset] [Sample Rate]
Timing Utils Tuple:	(rx_time {1432 0.912341 243123411 1000000.0})

Extending The 'Basic FSK Receiver' Example



Channel Filter

Quadrature Demod

Clock Recovery

Binary Slicer

BITSTREAM

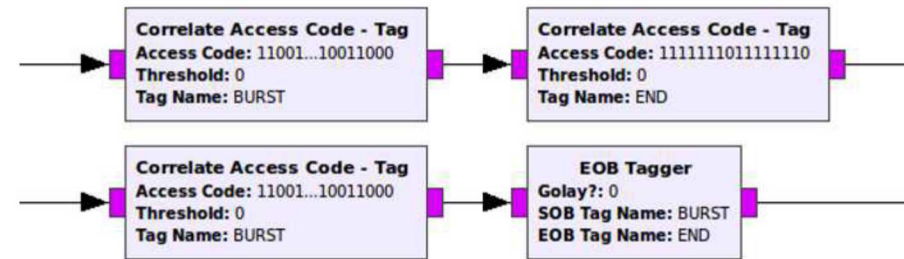
Burst Detection

GR Breaks down logical receiver operations into functional DSP Blocks

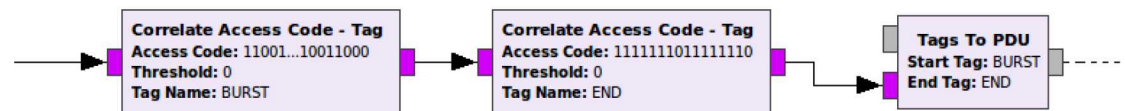
BURST: 0

Extending The 'Basic FSK Receiver' Example

- We have a bitstream and know where bursts start...now what?
- Encapsulate a complete burst of data
 - End-of-burst sequence detection
 - Encoded length field in header
 - Blind (fixed length after burst detection)



- Create a “protocol data unit” (PDU) with the detected data



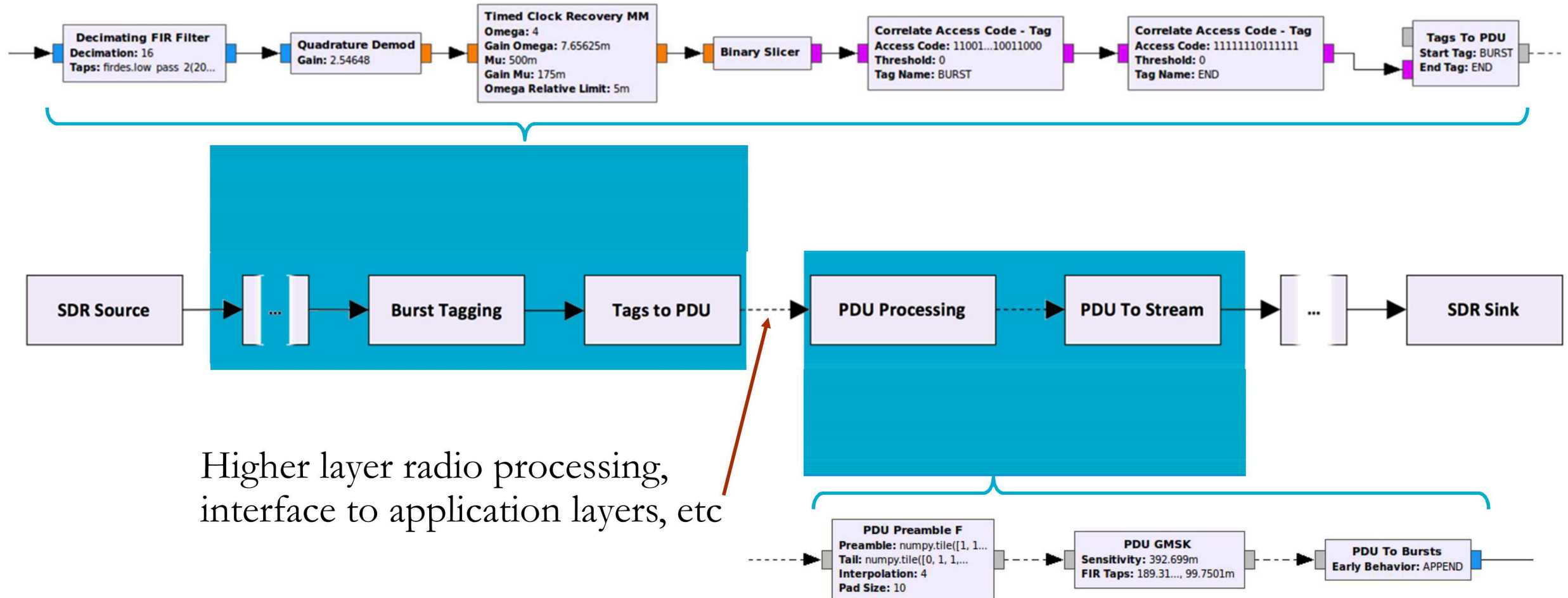
- Pass this information to a higher-layer processor

```
* MESSAGE DEBUG PRINT PDU VERBOSE *
((pdu_num . 1) (pdu_id . 1262624438) (time_type .
uhd_time_tuple) (burst_time . {0 0.103434}))
pdu_length = 72
contents =
0000: 00 00 01 00 01 01 00 01 01 01 00 01 00 01 00 00
0010: 00 00 00 00 01 01 01 01 00 01 01 00 01 00 00 01
0020: 01 00 01 01 00 00 00 01 00 01 01 01 00 00 01 01
0030: 00 01 00 01 00 00 00 01 00 01 00 00 01 00 00
0040: 00 00 00 00 01 01 01 01
```

Building a Simple TDMA FSK Modem



Building a Simple TDMA FSK Modem



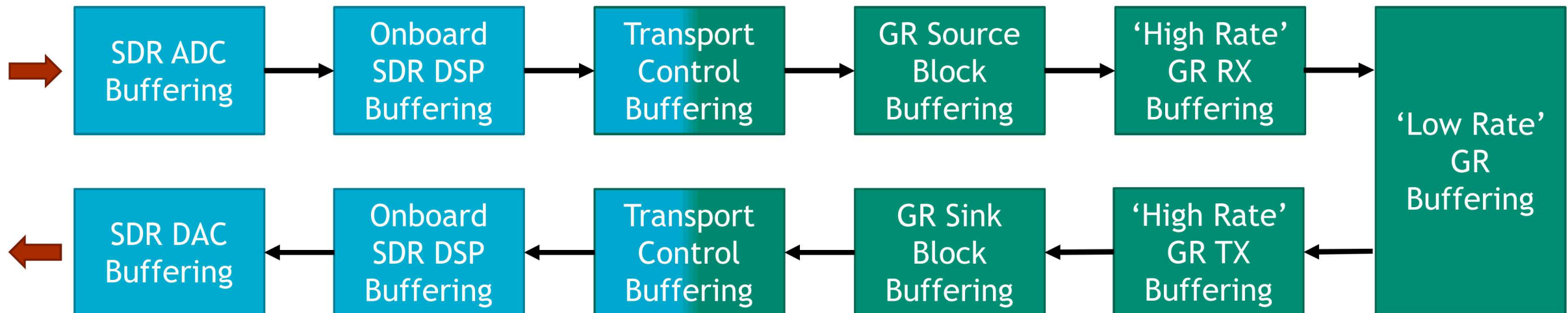
Higher layer radio processing,
interface to application layers, etc

“Low Latency”...?

- ‘Low latency’ is a key design consideration
 - Minimize time from burst RX to burst TX
- Blame the OS / Scheduler?
 - Its usually buffering...
 - Tradeoff...small buffers often means less efficient processing

$$TR_{latency_{min}} \geq \left(\frac{data_buffer_size}{sample_rate} \right)_{max}$$

- ✓ Reduce Data Buffer Sizes
- ✓ Increase Sample Rate
- ✗ More CPU Utilization

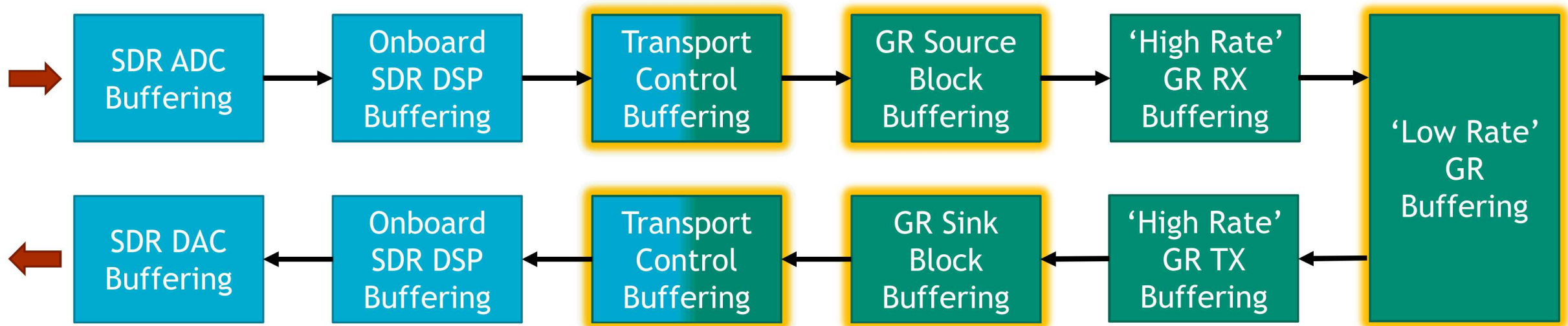


“Low Latency”...?

- ‘Low latency’ is a key design consideration
 - Minimize time from burst RX to burst TX
- Blame the OS / Scheduler?
 - Its usually buffering...
 - Tradeoff...small buffers often means less efficient processing

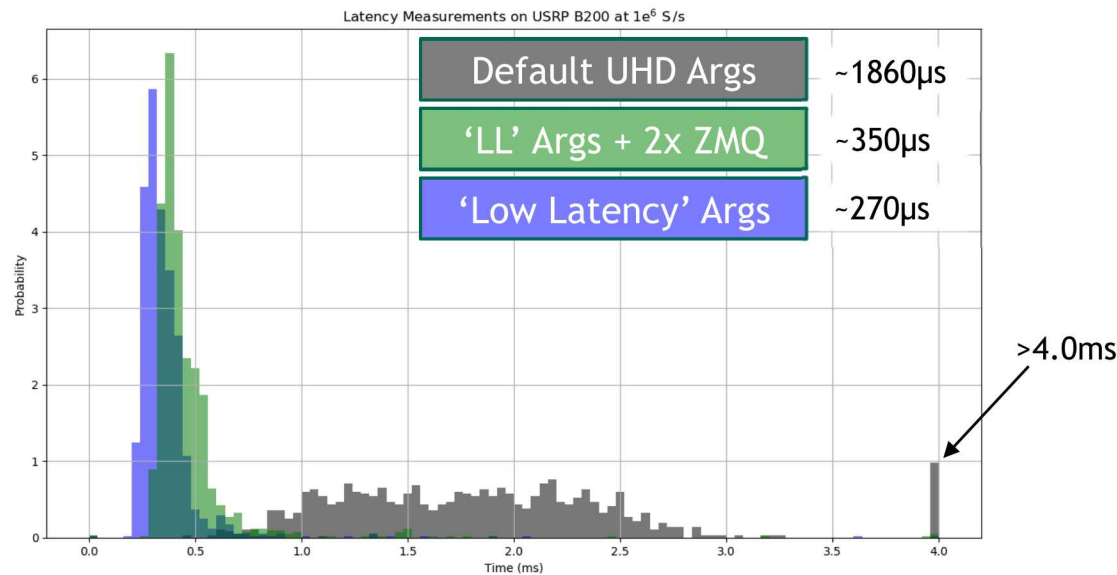
$$TR_{latency_{min}} \geq \left(\frac{data_buffer_size}{sample_rate} \right)_{max}$$

- ✓ Reduce Data Buffer Sizes
- ✓ Increase Sample Rate
- ✗ More CPU Utilization



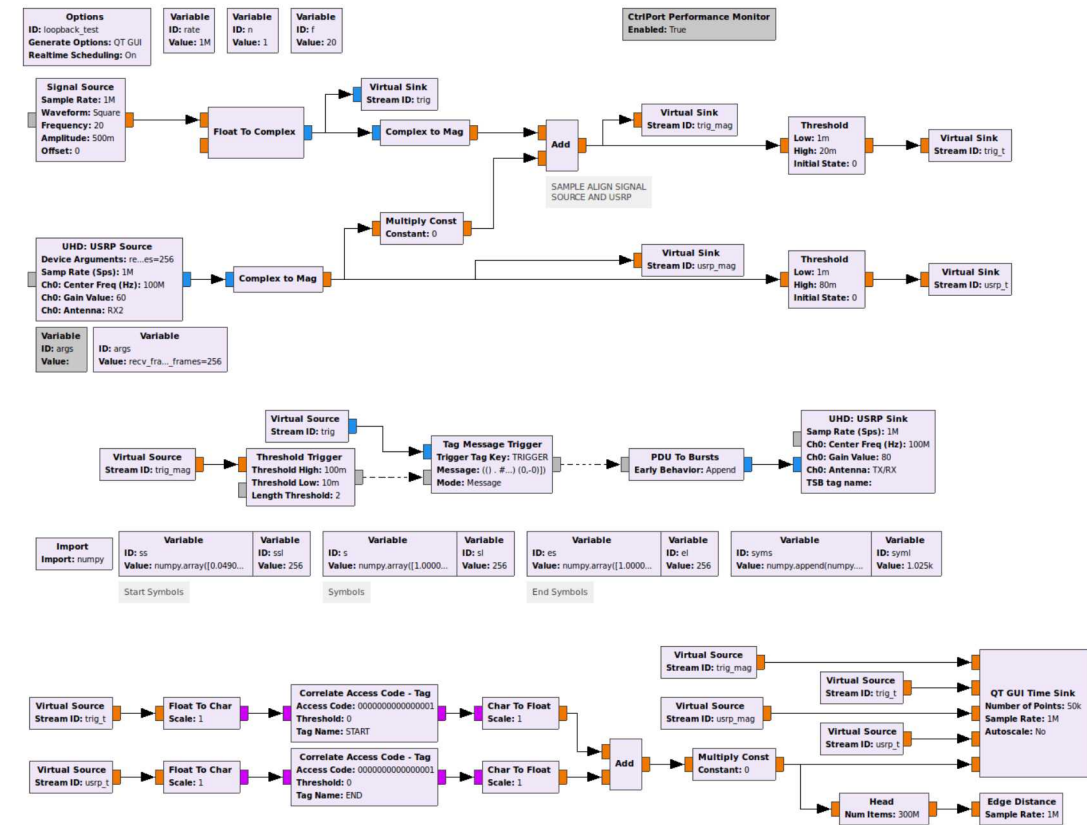
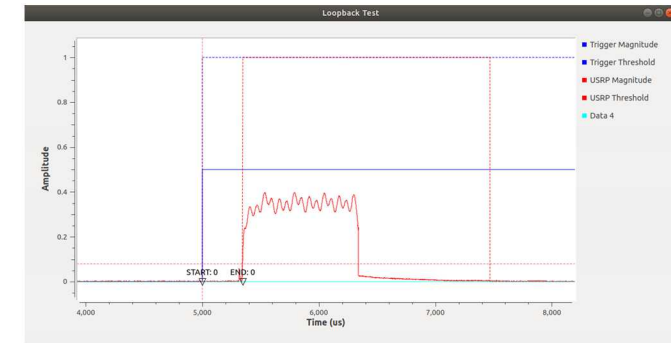
Characterizing Round Trip Latency

- Flowgraph included in Timing Utilities
- PULSE → RX → PDU → [...] → TX → SCOPE
latency requires external equipment
- TRIG → PDU → [...] → TX → RX → TIME SINK
is simpler (terminate TX port of USRP)



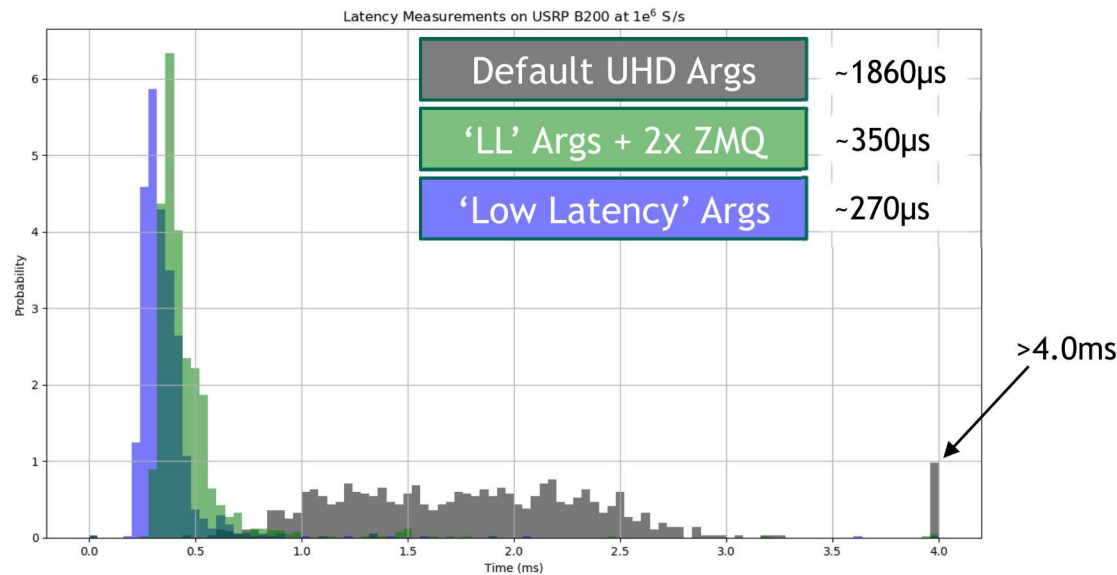
Low Latency UHD Arguments:

"recv_frame_size=256, num_recv_frames=256, send_frame_size=256, num_send_frames=256"



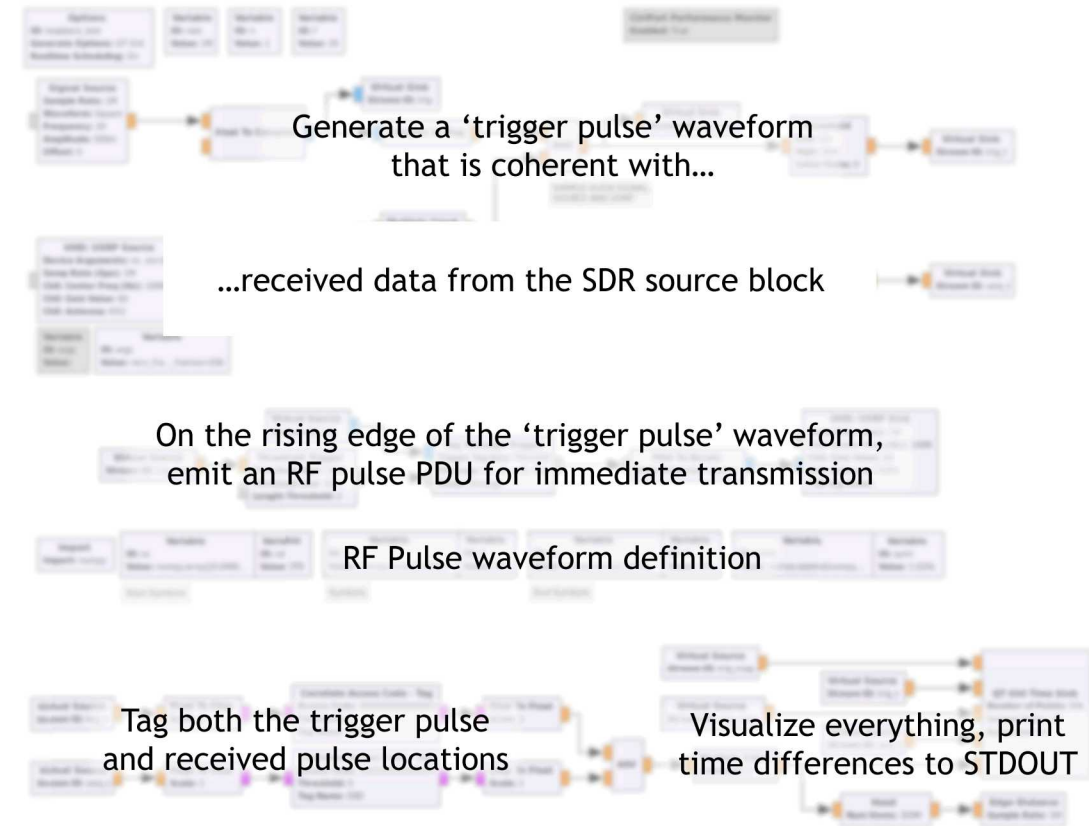
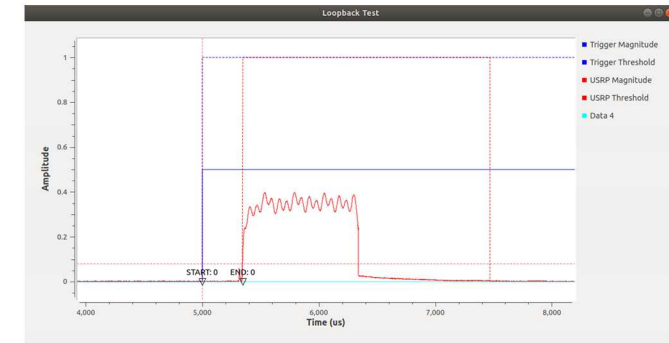
Characterizing Round Trip Latency

- Flowgraph included in Timing Utilities
 - PULSE → RX → PDU → [...] → TX → SCOPE
latency requires external equipment
 - TRIG → PDU → [...] → TX → RX → TIME SINK
is simpler (terminate TX port of USRP)



Low Latency UHD Arguments:

"recv_frame_size=256, num_recv_frames=256, send_frame_size=256, num_send_frames=256"



Notable Blocks in the PDU Utilities Module

○ PDU/Stream Conversion:

- Tags to PDU
- PDU to Bursts
- Take/Skip to PDU
- Tag Message Trigger

○ In-Tree Streaming Analog:

- Complex to Mag²
- Keep 1 in N
- Binary Arithmetic
- Up/Downsample
- PFB Arb Resampler
- FIR Filter
- FM Modulator
- Quadrature Demod

○ New PDU Manipulation Blocks:

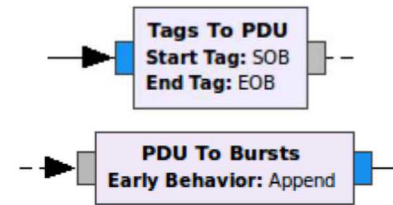
- Pack/Unpack
- PDU Alignment
- Commutator
- Data Encoding
- Extract Field
- PDU WPCR
- PDU Logger
- Flow Controller

○ Debugging Tools:

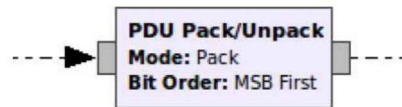
- PDU Counters
- QT PDU Source
- PDU Message Gate
- Random Drop

Notable Blocks

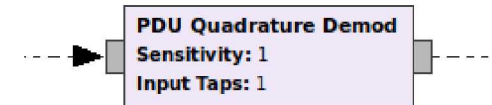
- PDU / Stream Conversion have been covered pretty well



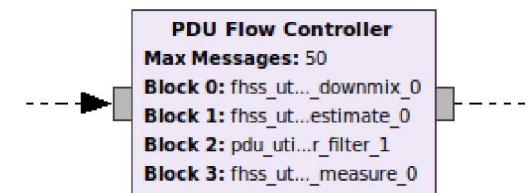
- Emits messages based on certain conditions
 - Robust arming/triggering architecture based on messages or stream tags
 - Re-trigger holdoff, arm timeout, TX limits, etc
 - Can emit fixed Messages or Timed PDUs



- Converts data within U8 PDU formats
 - Stuff U8 bits into U8 bytes, bit swap, etc
 - Requires work for higher order PDU translation



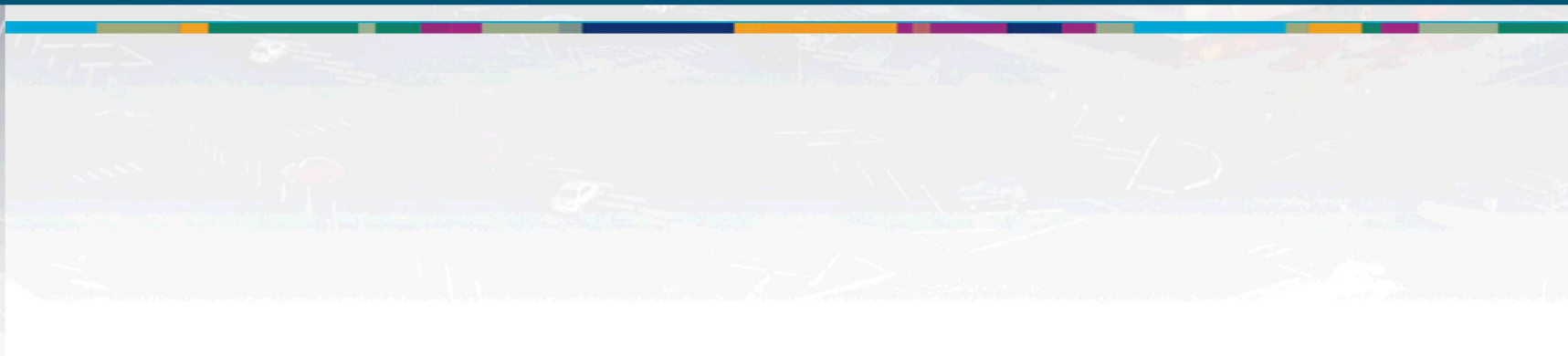
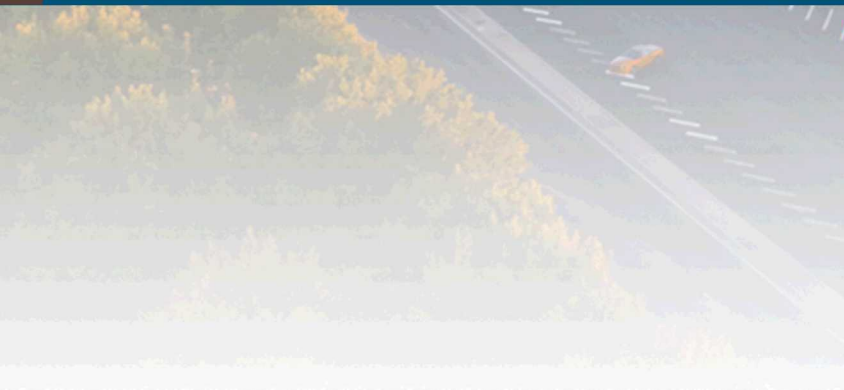
- Whole Packet Clock Recovery Symbol Synch
 - Based on M. Ossmann's WPCR project ^[1]
 - Operates well between 4 and 60 Samples/Symbol



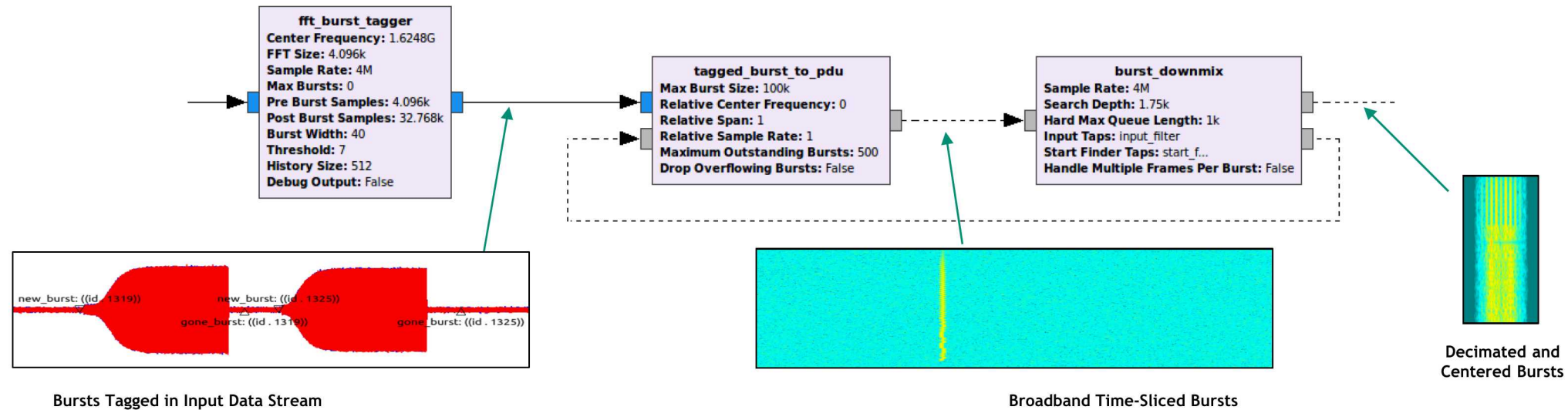
- Asynchronous message passing: no flow control
 - Block queues are checked, PDUs block if >Max
 - Helpful for highly variable input situations



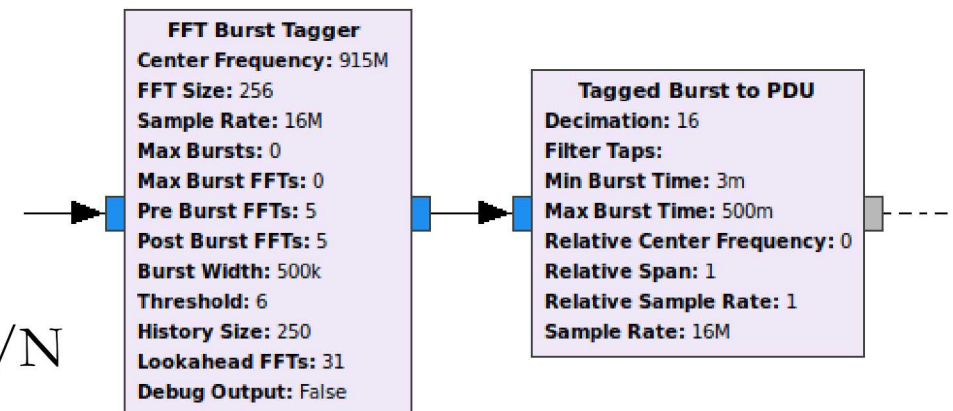
Other Sandia GR Modules



FHSS Utilities



- Built for ISM FHSS receiver applications
 - Bursts of energy on arbitrary frequencies
- Derived from CCC's gr-iridium
 - Similar approach, generalized
 - Efficient implementation, good sensitivity to $\sim 6\text{dB C/N}$

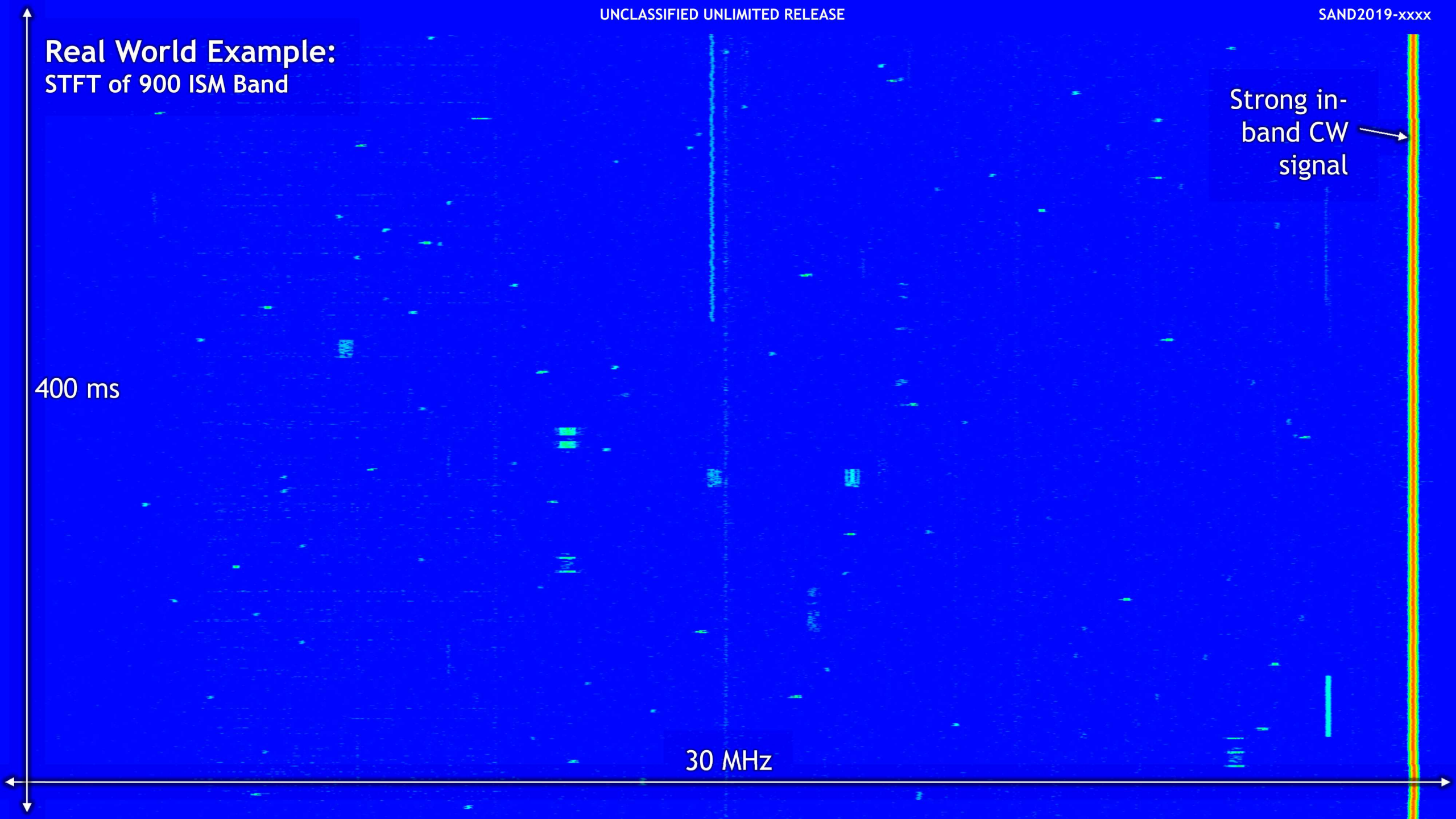


Real World Example: STFT of 900 ISM Band

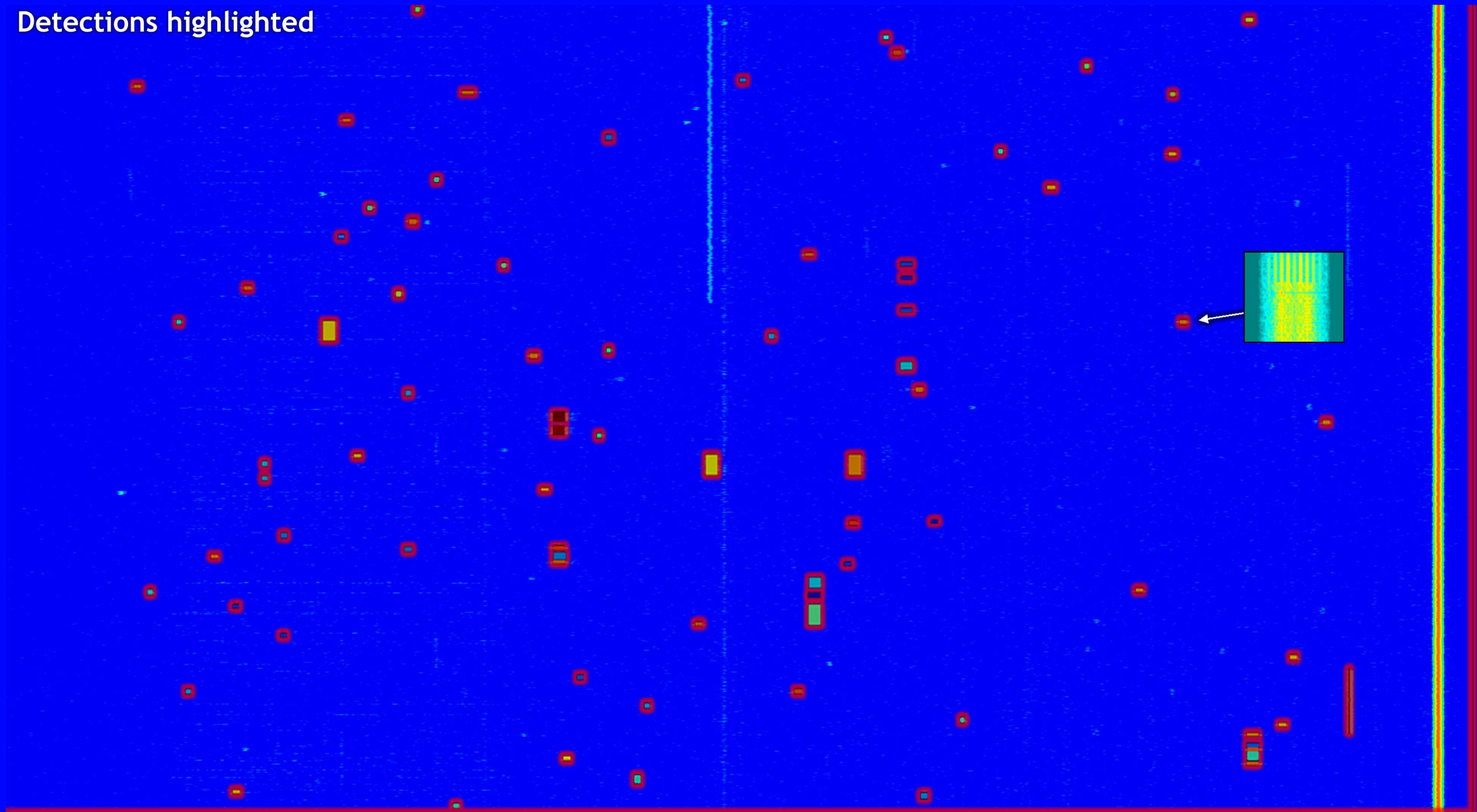
400 ms

30 MHz

Strong in-band
CW
signal

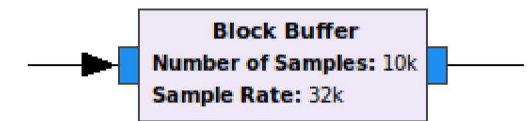
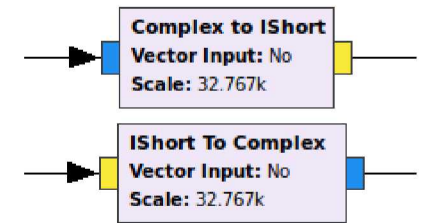


Detections highlighted



Sandia Utilities

- Assorted, generally useful blocks that don't fit well into other modules
 - Holding area for in-tree improvements we have yet to get released
 - Mostly oriented at debug applications
 - Usually minor documentation if any
 - Blocks are generally not extensively tested...may be dangerous
-
- Sometimes things get stuck in here before being moved or upstreamed...
 - Useful to install if you are using Sandia's GR Toolkit



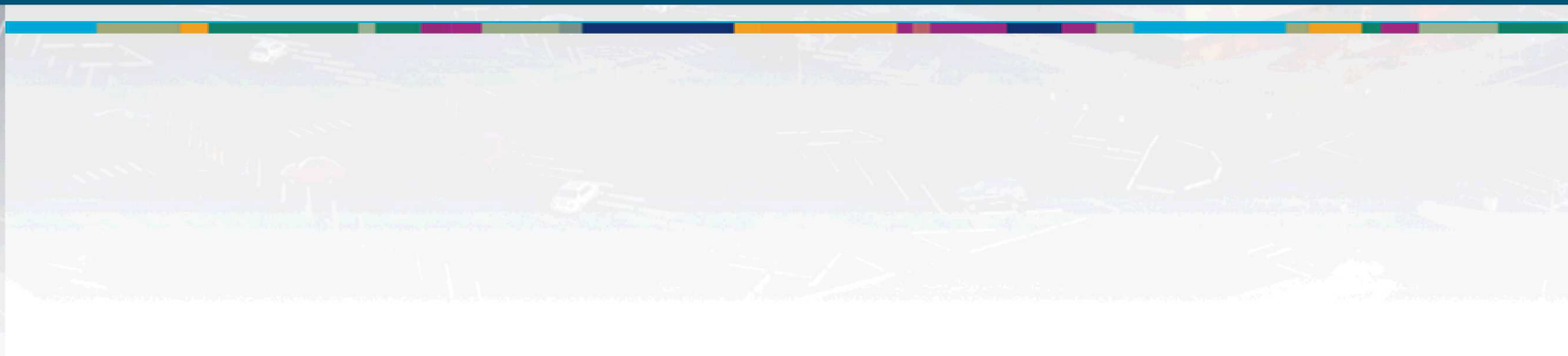
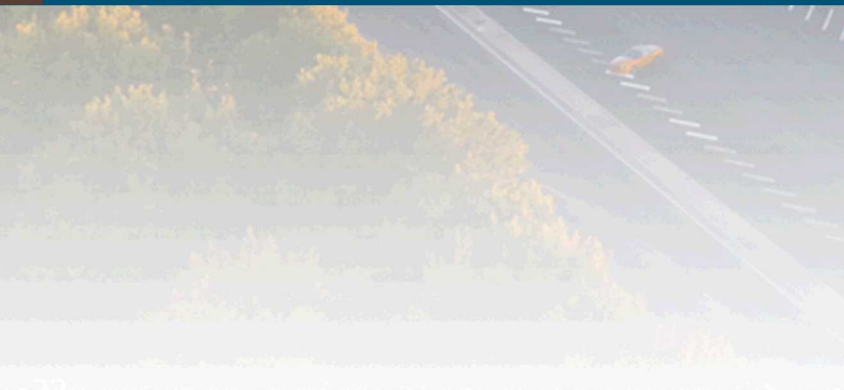
[Yet another] Sidekiq Module: gr-sidekiq_snl

- Coming Soon... in collaboration with Epiq Solutions
- Extends many UHD-style features to the Sidekiq radio line
 - DDC / DUCs in FPGA
 - Dual command queues in FPGA (instantaneous and timed commands)
 - Time tags generated on receive, support for TX tag based flow/time/frequency control
- Supports direct PDU-based transmission
- Very efficient implementation (thread-per-channel)
- Currently works on the M.2 and X2-VPX Sidekiq variants





Summary



Takeaways

- PDU's are underutilized, and very useful
 - With only *a bit* of frustrating PMT behavior...
- The PDU Utilities help bridge GNU Radio's streaming and PDU APIs
- There are straightforward tools available for developing bursty transceivers within GR
- Significant enhancement to GR's in-tree PDU capabilities are available
- GNU Radio can be used for 'reliable' 'low latency' applications
- The FHSS Utilities module has a robust and efficient arbitrary burst detector and associated signal processing

```
481 // OK, here comes the horrible part. Pairs pass is_dict(), but they're not dicts. Such
482 // dicks.
483 try {
484     // This will fail if msg is a pair:
485     pmt::pmt_t keys = pmt::dict_keys(msg);
486 } catch (const pmt::wrong_type& e) {
487     // So we fix it:
488     GR_LOG_DEBUG(d_debug_logger, boost::format("Converting pair to dict: %s") % msg);
489     msg = pmt::dict_add(pmt::make_dict(), pmt::car(msg), pmt::cdr(msg));
490 }
```